

Advanced NLP - 67664 - Exercise 1

Ofri Ahiel

May 31, 2026

Section 1 - Open Questions

Question 1:

Three QA datasets that serve as a format to annotate intrinsic linguistic properties:

1. **QA-SRL (Question-Answer Driven Semantic Role Labeling):** Annotates predicate-argument structures via QA pairs. It measures the intrinsic ability to identify semantic roles (e.g., who did what to whom) without requiring rigid linguistic formalism.
2. **QA-ZRE (Zero-Shot Relation Extraction):** Formulates relation extraction as reading comprehension. It tests the intrinsic property of understanding semantic relations between entities, evaluating generalization to unseen relation types.
3. **QADiscourse (Question-Answer Based Discourse Semantics):** Formulates the annotation of discourse relations between clauses as QA pairs. It measures the intrinsic ability to understand inter-sentence rhetorical connections and discourse structure, without relying on predefined discourse connective lexicons.

Question 2:

(a) Inference-time Scaling Methods Overview

Method	Description	Advantages	Computational Bottlenecks	Parallelizable?
Chain-of-Thought (CoT)	Generates intermediate reasoning steps explicitly before producing the final answer in a single continuous output.	Improves reasoning on math and logic tasks by eliciting step-by-step logic. Highly interpretable.	Sequence Length. More tokens strictly require more compute and increase KV cache memory overhead.	No. Token generation is strictly autoregressive.
Self-Consistency	Generates multiple distinct reasoning paths and aggregates the answers using a majority vote.	Robust against random reasoning errors. Improves accuracy without requiring parameter updates.	Compute & Memory. Requires massive compute (FLOPs) to sample multiple full paths.	Yes. Independent paths can be fully batched on a GPU.
Self-Ask	Decomposes a complex question into a sequence of intermediate sub-questions, solving each sequentially.	Excellent for compositional problems requiring specific, independent fact retrieval.	Latency. The model must sequentially generate intermediate questions and wait for their answers.	No. Inherently sequential process.
Verifiers	Uses external tools (unit tests, regex) or trained reward models at test time to score and filter reasoning paths.	Filters out flawed reasoning paths. Highly effective for domains with objective truths.	Memory & Compute. Requires overhead to run verification functions over multiple candidate outputs.	Partially. Path generation and scoring can be batched across candidates.
Self-Evaluation	The model generates an initial solution, evaluates its own output, and revises it based on detected errors.	Enables adaptive problem-solving and dynamic error correction during the inference phase.	Expensive Inference. Long traces due to the iterative generate-critique-refine loop.	No. Evaluation strictly depends on the initial generation.

(b) Method Selection for a Complex Scientific Task

Given access to a single GPU with a large memory capacity for a complex scientific task, I would choose to implement **Self-Consistency**. The primary hardware bottleneck of a single GPU is the latency introduced by sequential generation (as seen in methods like Self-Ask or Self-Evaluation), which leaves most of the GPU’s compute units idle. However, a large memory capacity provides the ideal environment to leverage massive batching. Because the sampling of distinct reasoning paths in Self-Consistency involves no data dependencies between sequences, dozens of independent paths can be generated simultaneously in a single large batch. This approach fully saturates the GPU’s compute cores and maximizes the available memory, yielding the performance gains of inference-time scaling without significantly increasing the overall wall-clock time compared to a single greedy decode. Furthermore, from a task perspective, scientific reasoning requires high precision and is highly susceptible to logical missteps in a single generation pass. By sampling multiple diverse reasoning paths and taking a majority vote to marginalize out random errors, Self-Consistency directly mitigates hallucinations and provides the structural robustness required for complex scientific problem-solving.

Section 2 - Programming Exercise

GitHub Repository

https://github.com/OfriA/AdvancedNLP67664_EX1

Hyperparameter Search Setup

For the hyperparameter search, I fine-tuned the `bert-base-uncased` model on the MRPC dataset. I executed three different runs using the following configurations:

- **Run 1:** Learning Rate = $2e-5$, Batch Size = 16, Epochs = 3 $\rightarrow$ Validation = 86.03%
- **Run 2:** Learning Rate = $3e-5$, Batch Size = 32, Epochs = 3 $\rightarrow$ Validation = 84.07%
- **Run 3:** Learning Rate = $5e-5$, Batch Size = 16, Epochs = 4 $\rightarrow$ Validation = 87.01%

Run 3 performed the best on the validation set at the end of training, achieving an accuracy of 87.01%, while Run 1 and Run 2 achieved lower validation accuracies of 86.03% and 84.07%, respectively. As shown in Figure 1, the training loss for all three configurations decreased steadily over time, confirming successful fine-tuning. The visual plot distinctly reflects the hyperparameter choices: the configuration with the larger batch size of 32 (light blue) completed its optimization in fewer total steps, while the configuration with the highest learning rate of $5e-5$ and more epochs (green) ran for the most steps and exhibited the highest volatility.

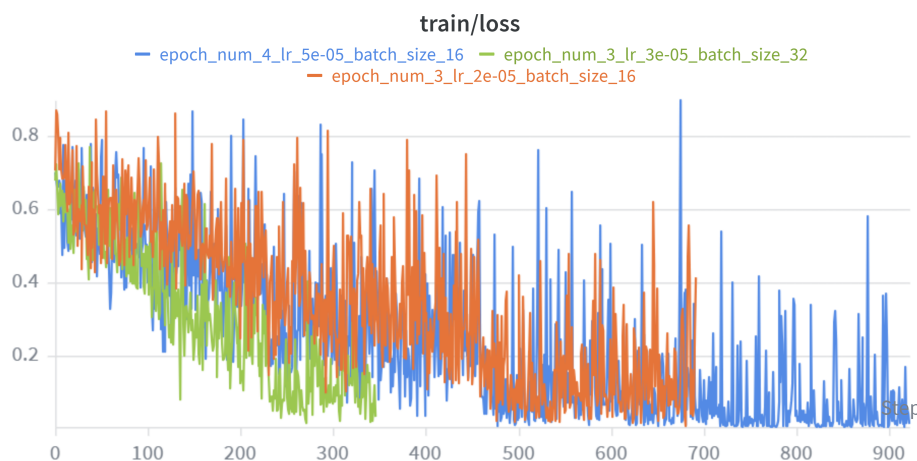


Figure 1: Training loss over time for the three evaluated hyperparameter configurations.

Test Accuracy Comparison

After evaluating the runs, I used the saved checkpoints to make predictions on the test set. Interestingly, the configuration that achieved the highest validation accuracy did not achieve the highest test accuracy. While Run 3 was the optimal model during validation (87.01%), it suffered a significant drop on unseen data, achieving the lowest test accuracy of 82.61%. In contrast, Run 1, which had a middle-tier validation accuracy of 86.03%, generalized the best to the unseen data, achieving the highest test accuracy of 83.83%. Furthermore, Run 2 achieved a test accuracy of 83.48% despite having the lowest validation score (84.07%). This highlights how models can easily overfit to the validation set during hyperparameter tuning, and that the top validation score does not always guarantee the best real-world generalization.

Qualitative Analysis

To understand where the lower-performing configurations failed, I compared the validation predictions of the best validation configuration (Run 3) against the weaker models. The lower-performing models consistently struggled with sentences that possessed high lexical overlap but contained subtle semantic

contradictions or detail mismatches. Below are specific validation examples where the optimally tuned model correctly predicted the label, but at least one of the weaker models failed:

[Example 1]

- **Sentence 1:** While dioxin levels in the environment were up last year , they have dropped by 75 percent since the 1970s , said Caswell .
- **Sentence 2:** The Institute said dioxin levels in the environment have fallen by as much as 76 percent since the 1970s .
- **True Label:** 0 (Not Paraphrase)
- **Predictions:** Best Model (Run 3) = 0 | Weaker Models = 1, 1

Both weaker models falsely predicted a paraphrase due to the heavy phrase overlap ("dioxin levels in the environment", "since the 1970s"). They failed to capture the conflicting numerical statistics (75% vs. 76%) and the difference in attribution ("said Caswell" vs. "The Institute said").

[Example 2]

- **Sentence 1:** In midafternoon trading , the Nasdaq composite index was up 8.34 , or 0.5 percent , to 1,790.47 .
- **Sentence 2:** The Nasdaq Composite Index .IXIC dipped 8.59 points , or 0.48 percent , to 1,773.54 .
- **True Label:** 0 (Not Paraphrase)
- **Predictions:** Best Model (Run 3) = 0 | Weaker Model (Run 1) = 1

The Run 1 model over-indexed on the shared financial entities ("Nasdaq composite index") but entirely missed the contradictory directional verbs ("was up" vs. "dipped"), falsely classifying them as equivalent.

[Example 3]

- **Sentence 1:** The driver , Eugene Rogers , helped to remove children from the bus , Wood said .
- **Sentence 2:** At the accident scene , the driver was " covered in blood " but helped to remove children , Wood said .
- **True Label:** 0 (Not Paraphrase)
- **Predictions:** Best Model (Run 3) = 0 | Weaker Model (Run 2) = 1

The Run 2 model failed to recognize that while the core action ("helped to remove children") is shared, the sentences convey different specific details (the driver's name vs. being "covered in blood"). The best configuration demonstrated a more robust semantic understanding, correctly classifying the underlying divergence in meaning.